Pessimistic-Behavior / Greedy-Target Q-Learning against Adversarial State Perturbations

Adithya Venkatesh (independent researcher)
Zizhan Zheng -- Department of Computer Science, Tulane University
zzheng3@tulane.edu

ArXiv v1 draft for advisor review. Compile main.tex on Overleaf for camera-ready math.

Abstract

A well-trained RL agent can be defeated by strategically perturbing its state observations at test time. Sun and Zheng (ICLR 2024) address this with belief-enriched pessimistic Q-learning: maximin actions over a belief, used both when collecting data and in Bellman targets. That defense is strong in discrete domains but costly at test time. We study a cheaper variant, PB-GT (Pessimistic-Behavior / Greedy-Target): training still uses pessimistic (maximin) behavior under perturbed observations, but the target is ordinary greedy Q-learning on the true next state, and test actions are greedy on the observed state -- no belief, no maximin. On tabular 5x5 GridWorld (500 seeds), full pessimistic Q-learning remains far more robust under trained DQN attackers (-21.8 +/- 50.0 vs -552.4 +/- 335.2 for PB-GT and -682.1 +/- 329.0 for vanilla), while PB-GT matches vanilla inference speed and is ~11.6x faster than maximin. A Double-DQN port to CartPole-v1 (5 seeds) is preliminary: vanilla converges after checkpoint restore, but BP-style pessimistic and PB-GT do not reproduce the GridWorld ordering. We present PB-GT as a fast approximation with a documented robustness gap, not a substitute for BP-DQN.

1. Introduction

RL agents that act on learned value networks are brittle under adversarial state perturbations: a test-time attacker, subject to budget eps, replaces true state s by a nearby observation s~ in B_eps(s) to minimize return (Huang et al. 2017; Zhang et al. 2020). The attacker does not change transition dynamics -- only what the agent sees. Defenses include SA-MDP / SA-DQN and WocaR (regularize smoothness) and ATLA (co-train an attacker).

Sun and Zheng (ICLR 2024) view the problem as a Stackelberg Markov game and propose pessimistic Q-learning: pi(s~) = argmax_a min_sbar Q(sbar, a), used in both behavior and targets. Belief M_t (tabular history, or PF-RNN particles in BP-DQN) shrinks uncertainty; DP-DQN uses diffusion on pixels. Maximin-over-belief is the source of robustness and of cost: ~11x slower than greedy on our GridWorld, ~50x on CartPole DQN.

Question. If we keep pessimistic behavior while collecting data, but revert to vanilla greedy targets and greedy test actions, how much robustness do we retain, and how much inference cost do we save?

PB-GT is that pairing -- not 'off-policy DQN' (all three methods are off-policy). It is a deliberate train/test mismatch, opposite to Sun and Zheng's consistency argument.

Contributions. (1) Tabular reproduction at scale: vanilla vs Algorithms 1-2 on 5x5 GridWorld, 500 seeds, trained DQN attackers. (2) PB-GT: pessimistic behavior, greedy target, greedy test. (3) Preliminary CartPole Double-DQN port (BP-style GRU belief, not full PF-RNN). (4) Scoped claim: v1 documents the robustness-compute tradeoff; closing the gap to BP-DQN is future work.

2. Methods (summary)

	Vanilla	**Pessimistic**	**PB-GT**
Train beh.	eps-greedy on s	maximin on s~	same as pes.
TD target	greedy on s'	maxmin over M'	greedy on s'
Test (atk.)	greedy on s~	maximin over M	greedy on s~
Belief	none	yes	train only

Paper	DQN	BP-style	this work

Vanilla: tabular Q or Double DQN; test greedy on the observation. Pessimistic (tabular): Sun and Zheng Algorithms 1-2. Discrete ball = cell + 1-step neighbors. When |M intersect B|>1, our policy uses mean Q over the belief (softer approximation); TD target still uses maximin. Fine-tune 500 episodes from a vanilla Q-table.

PB-GT: same collection as pessimistic (omega_pi, eps-greedy maximin, belief update) but backup is vanilla greedy on true s'. Test: a = argmax Q(s~, a) -- no belief. Inference = vanilla. Deep PB-GT DQN fine-tunes from vanilla (often freeze backbone): PGD s~, maximin behavior over GRU particles, Double-DQN greedy targets on true s', greedy test. Full pseudocode: main.tex appendix.

3. Setup

GridWorld: 5x5; step -1; gold (0,3) +10; bomb (4,3) -10; gamma=1; vanilla 2000 episodes; pes/PB-GT 500 fine-tune from same Q; eval 100 episodes; cap 1000 steps; n=500 seeds. Attackers: SB3 DQN, frozen; vanilla vs pessimistic attacker; pes/PB-GT vs own attackers. Not MinBest / PA-AD / iterative PGD.

CartPole-v1 (preliminary): eps=0.05 L_inf raw; vanilla Double DQN [128,128], 500k steps, restore-best (solve >=475). Belief: GRU h=64, kappa=20, 30k pretrain (not kappa_p=30 PF-LSTM). Pes fine-tune: freeze backbone, LR 5e-5, <=100k, maximin target. PB-GT: 40k steps, LR 1e-5, greedy target. n=5. Hardware: M1 Pro CPU (Sun and Zheng used RTX 3090). Metrics: clean; matched SB3 attacker (81 actions, 80k steps); shared random delta; PGD* = 32-candidate search.

4. Results

4.1 Tabular GridWorld (main result)

Method	Clean	Under attack	Atk. infer. (ms)
Vanilla	7.35 +/- 0.24	-682.1 +/- 329.0	0.008
Pessimistic	7.35 +/- 0.27	-21.8 +/- 50.0	0.093
PB-GT	7.37 +/- 0.23	-552.4 +/- 335.2	0.008

Clean returns are essentially identical. Under attack: pessimistic >> PB-GT > vanilla. Pessimistic beats PB-GT in 96.4% of seeds and vanilla in 96.0%; PB-GT beats vanilla in 60.6%. PB-GT is a modest improvement over vanilla, not a near-match to maximin. Inference: 11.6x slower for pessimistic vs PB-GT/vanilla.

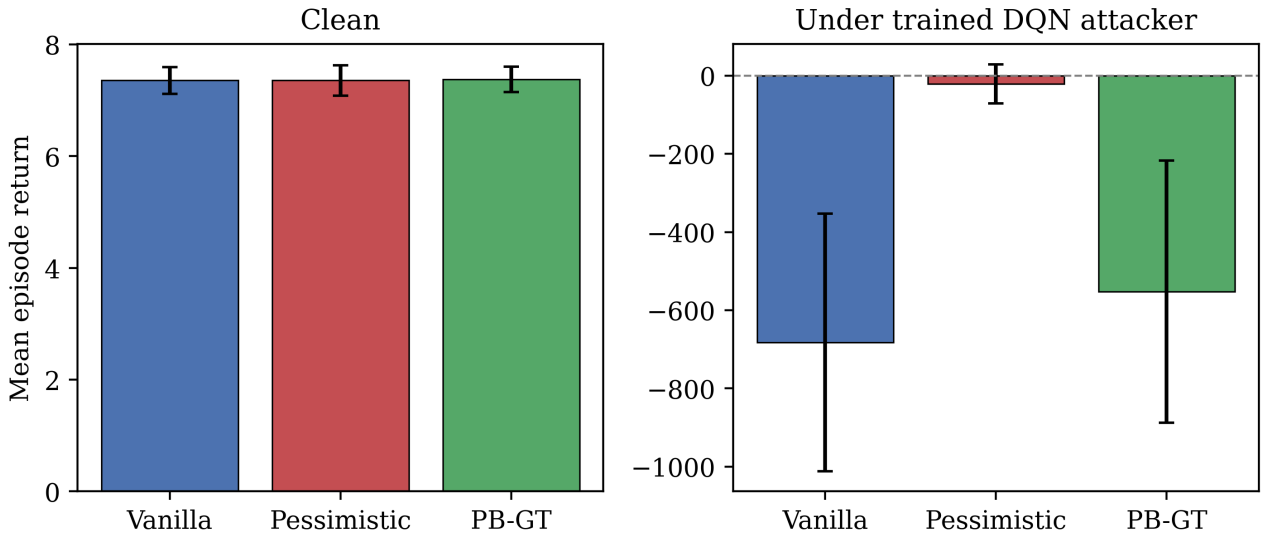


Figure 1. GridWorld mean return +/- std (n=500).

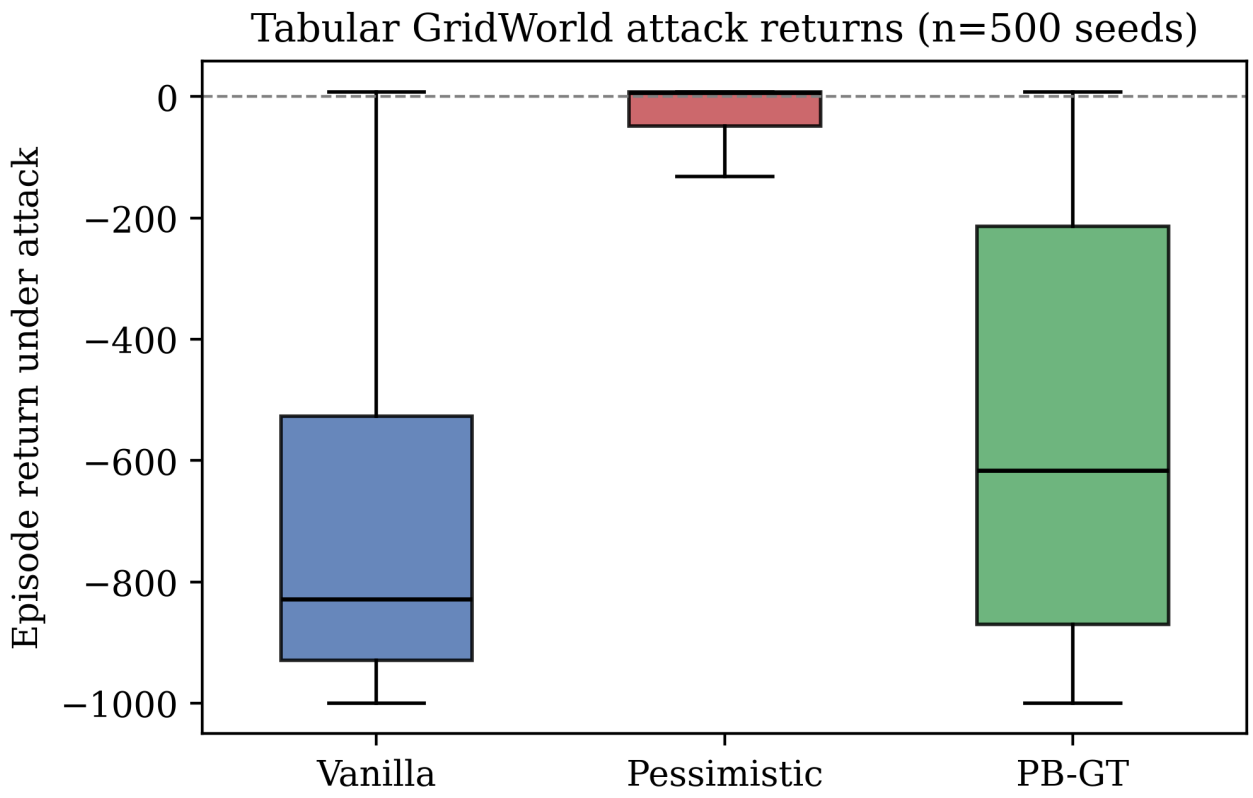


Figure 2. Attack-return distributions across 500 seeds.

4.2 Inference cost

Environment	Pes (ms)	PB-GT (ms)	Slowdown
GridWorld	0.093	0.008	11.6x
CartPole (matched)	1.88	0.037	51x

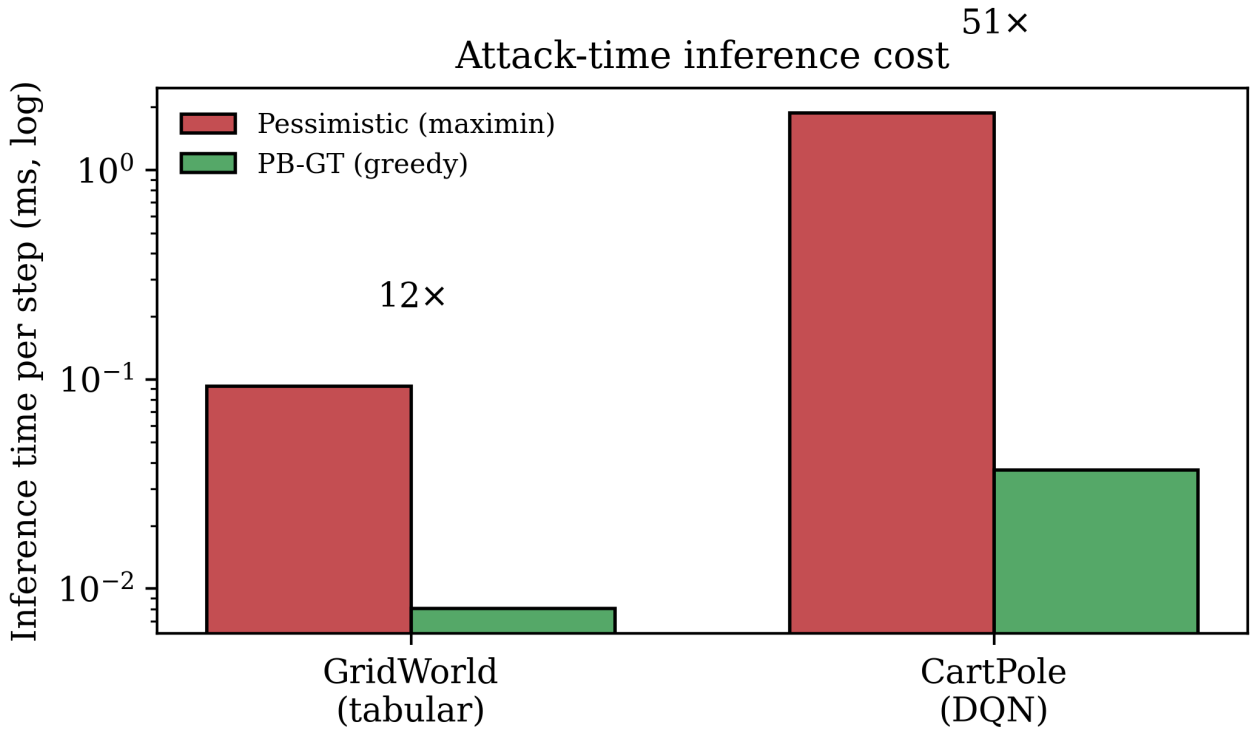


Figure 3. Attack-time ms/step (log scale).

4.3 CartPole-v1 (preliminary -- do not oversell)

Metric	Vanilla	Pes (BP-style)	PB-GT
Clean	499 +/- 2	334 +/- 226	403 +/- 214
Matched attacker	113 +/- 216	11 +/- 3	109 +/- 219
Shared random d	284 +/- 176	125 +/- 185	150 +/- 209
PGD*	319 +/- 154	131 +/- 126	189 +/- 148

Vanilla: 5/5 seeds solved after restore-best (499 +/- 2). Pes/PB-GT unstable on clean; pes ~11 on every seed under matched attack. No deep robustness claim. Per-seed clean (V/P/PB-GT): 0: 500/500/500; 1: 500/133/500; 2: 500/500/500; 3: 496/44/20; 4: 500/490/495.

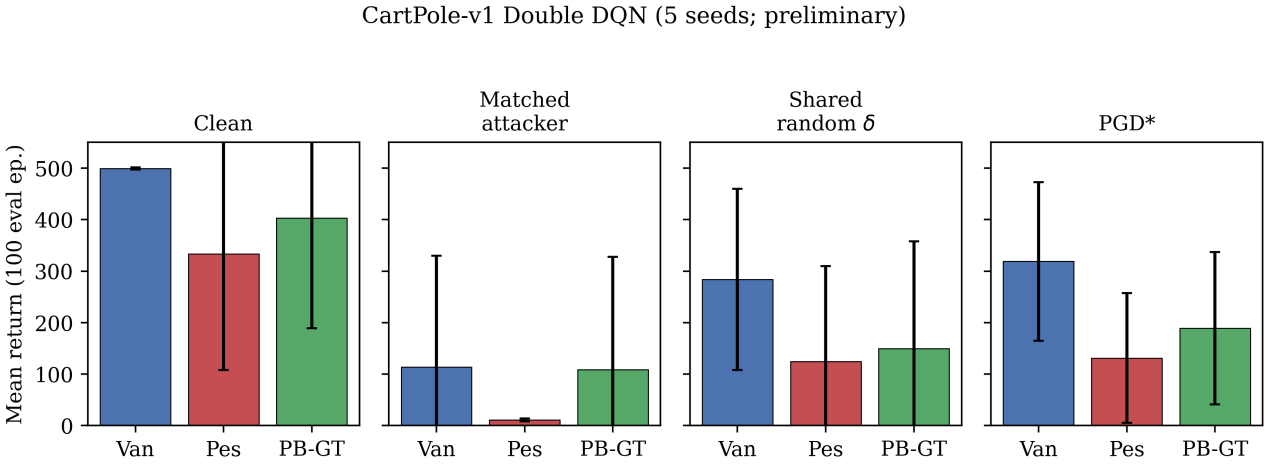


Figure 4. CartPole metrics (n=5). Treat as preliminary.

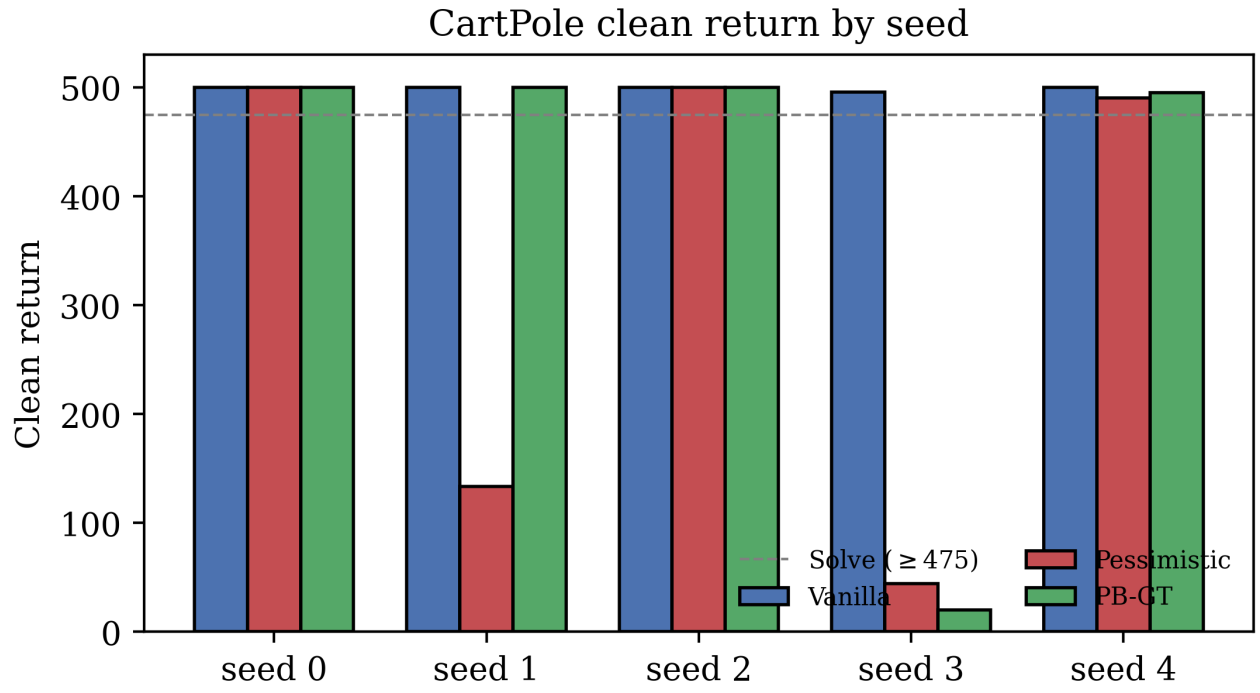


Figure 5. CartPole clean return by seed.

5. Limitations and future work

Not shown: PB-GT approaching pes/BP-DQN robustness; CartPole confirmation; Xiaolin Table 1a (continuous GridWorld); CAR-DQN / SA-DQN / WocaR / Atari; iterative PGD; vanilla-matched GridWorld attacker. Next: (1) continuous GridWorld + official BP-DQN, then PB-GT; (2) CAR-DQN baseline; (3) PB-GT refinements (hybrid inference, distillation, mixed targets); (4) Atari + DP-DQN on GPU.

6. Conclusion

Vanilla vs pessimistic vs PB-GT on 500-seed GridWorld and 5-seed CartPole DQN: full pessimism is far more robust under learned attacks; PB-GT is far cheaper at test time and only slightly better than vanilla. The deep port is not yet a robustness result. v1 claim: the robustness-compute tradeoff of belief-maximin Q-learning is real; PB-GT is a well-defined fast baseline that does not close the gap. Closing that gap on Sun and Zheng's environments is the next paper.

References

Huang et al. Adversarial attacks on neural network policies. ICLR Workshop, 2017.

Ilahi et al. Challenges and countermeasures for adversarial attacks on DRL. IEEE TAI, 2022.

Li et al. Towards optimal adversarial robust Q-learning (CAR-DQN). arXiv:2402.02165, 2024.

Liang et al. Worst-case-aware robust RL (WocaR). NeurIPS, 2022.

Ma et al. Particle filter recurrent neural networks. AAAI, 2020.

Mnih et al. Human-level control through deep RL. Nature, 2015.

Sun and Zheng. Belief-enriched pessimistic Q-learning. ICLR, 2024.

van Hasselt, Guez, Silver. Deep RL with double Q-learning. AAAI, 2016.

Watkins and Dayan. Q-learning. Machine Learning, 1992.

Zhang et al. Robust DRL against adversarial perturbations on state observations. NeurIPS, 2020.

Zhang et al. Robust RL on state observations with learned optimal adversary. ICLR, 2021.

Code / data

main_with_vanilla.py, gridworld_experiments_500runs.csv, main_cartpole_dqn.py, cartpole_belief.py, cartpole_experiments_restore_5runs.csv. LaTeX source: arxiv_v1/main.tex. BP-DQN code: github.com/SliencerX/Belief-enriched-robust-Q-learning. CAR-DQN: github.com/leoranlmia/CAR-DQN.